



FINAL PROJECT · TECHNICAL REPORT

# RentTitan

A production-oriented full-stack JavaScript MVP

*Explainable rental-dossier scoring through microservices*

---

React · Express · PostgreSQL · MongoDB · REST · GraphQL · OAuth 2.0

Prepared by

**Adam Hamdi · Selim Abouleila · Anaïs Robert**

July 2026

[github.com/Selim-Abouleila/RentTitan](https://github.com/Selim-Abouleila/RentTitan)

SECTION 01 · FINAL PROJECT REPORT

Executive summary

A rubric-complete local MVP with explicit service boundaries and an explainable product core.

RentTitan helps rental applicants understand and present the strength of their dossier. After signing in with Google, a user records target rent, monthly income, employment status and guarantors; tracks five document categories; receives a deterministic score out of 100 with missing-item indicators and improvement suggestions; and can generate a professional French landlord introduction through Google Gemini.

The implementation separates identity and structured financial data, document-checklist state, and score aggregation across a React frontend, one main Express backend and two Express microservices. PostgreSQL with Prisma stores users, dossiers and guarantors. MongoDB with Mongoose stores a per-user checklist. REST and GraphQL satisfy the two-paradigm requirement, while bearer JWTs secure protected application operations and are forwarded between services.

|                 |           |               |               |                |
|-----------------|-----------|---------------|---------------|----------------|
| 3               | 2         | 2             | 5             | 1              |
| SERVER PROJECTS | DATABASES | API PARADIGMS | SCORE SIGNALS | AI INTEGRATION |

What the project delivers

- **An interactive JavaScript single-page application** with a protected dashboard.
- **A main backend plus two independently owned microservices.**
- **Two persistent data stores**, Google OAuth, JWT access control, REST, GraphQL and a Gemini integration.

**ACCURATE POSITIONING** RentTitan is a complete, production-oriented educational MVP. It is not yet production-deployed: automated tests, CI/CD, application containerization, observability, secure file storage and hardened browser-session handling remain next-phase work.

## Product experience

One guided flow turns an uncertain rental file into a score, an explanation and a next action.

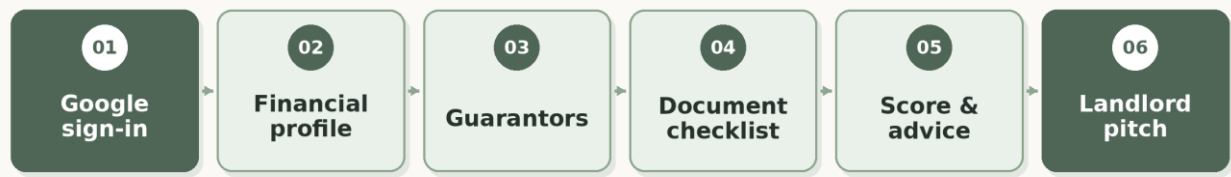


Figure 1. End-to-end application journey implemented by the React client and three server-side projects.

### Core user value

| Capability         | User outcome                                                   |
|--------------------|----------------------------------------------------------------|
| Google sign-in     | Fast, identifiable onboarding through OAuth 2.0.               |
| Financial dossier  | Centralizes rent, income, employment and multiple guarantors.  |
| Document checklist | Makes five completeness categories visible and actionable.     |
| Explainable score  | Returns a result, missing documents and concrete improvements. |
| AI landlord pitch  | Converts real profile data into a concise French message.      |

### MVP boundary

**DOCUMENT HANDLING** The upload flow is deliberately simulated. Multer receives the file in memory, the bytes are discarded, and MongoDB stores checklist state or generated dummy filenames only. The report therefore does not claim encrypted file storage or document verification.

The interface is designed for the live demo: login → dossier → checklist → score → AI pitch. The repository contains UI source rather than committed screenshots, so the application itself should be shown during the presentation.

## System architecture

Distributed responsibility without hiding the real client-to-service communication pattern.

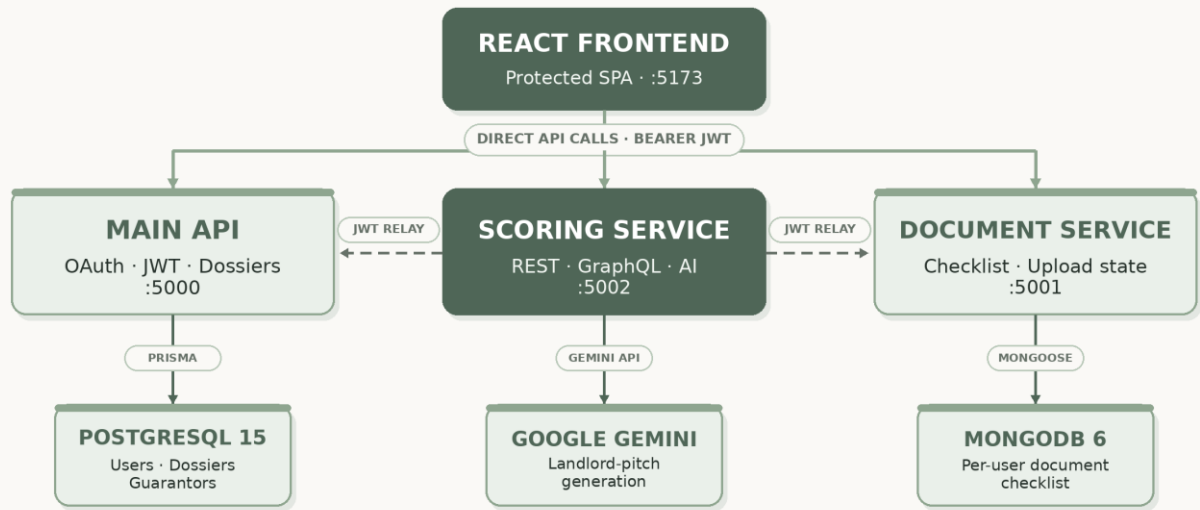


Figure 2. Runtime topology. The frontend calls all three backend projects; the scoring service aggregates data with JWT forwarding.

| Component      | Technology                       | Responsibility                                    |
|----------------|----------------------------------|---------------------------------------------------|
| Frontend: 5173 | React 19, Vite, Router, Tailwind | Login, protected UI, forms, score and pitch       |
| Backend: 5000  | Express 5, Passport, Prisma      | OAuth, JWT, dossier and guarantor REST API        |
| Document: 5001 | Express, Multer, Mongoose        | Checklist state and simulated upload/remove flows |
| Scoring: 5002  | Express, Apollo GraphQL, Gemini  | Score aggregation, recommendations and AI pitch   |

## Services and data ownership

Two databases are used for distinct models rather than as duplicate persistence layers.

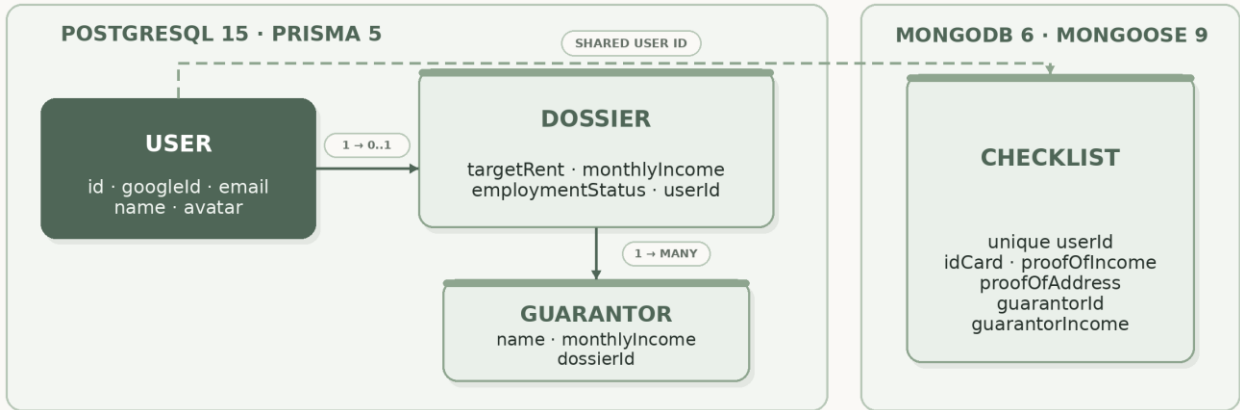


Figure 3. PostgreSQL relationships and the MongoDB checklist keyed by the authenticated user ID.

| Persistence    | Access layer          | Owned data                          | Owner            |
|----------------|-----------------------|-------------------------------------|------------------|
| PostgreSQL 15  | Prisma 5              | User, Dossier, Guarantor            | Main backend     |
| MongoDB 6      | Mongoose 9            | Checklist flags and filename arrays | Document service |
| No local store | Stateless aggregation | Computed score and suggestions      | Scoring service  |

### Data lifecycle

- Google profile data creates or resolves the User record in PostgreSQL.
- Dossier updates replace guarantors and return the complete relational aggregate.
- Checklist updates are isolated in MongoDB and uniquely keyed by userId.
- The score is calculated on demand; no score history is persisted.

**INFRASTRUCTURE BOUNDARY** docker-compose.yml starts PostgreSQL and MongoDB only. The four application processes are started separately; no application Dockerfiles or one-command production stack are committed.

## SECTION 05 · FINAL PROJECT REPORT

## APIs and communication

REST handles resources and actions; GraphQL exposes the score projection used by the dashboard.

| Method        | Endpoint                  | Service  | Access             |
|---------------|---------------------------|----------|--------------------|
| GET           | /auth/google              | Main API | Public OAuth start |
| GET           | /auth/google/callback     | Main API | Google callback    |
| GET           | /auth/protected-demo      | Main API | JWT                |
| GET /<br>POST | /dossiers                 | Main API | JWT                |
| GET           | /status                   | Document | JWT                |
| POST          | /upload                   | Document | JWT + multipart    |
| DELETE        | /documents/:documentType  | Document | JWT                |
| GET           | /api/v1/dossier-score     | Scoring  | JWT                |
| POST          | /graphql                  | Scoring  | Token forwarded    |
| POST          | /api/v1/ai/generate-pitch | Scoring  | JWT                |

### GraphQL projection

```
query MyDossier {
  myDossier { score missingDocuments suggestions }
}
```

### API-as-a-service demonstration

| Postman call            | Expected | Response contract                        |
|-------------------------|----------|------------------------------------------|
| Valid Bearer JWT        | 200      | { score, missingDocuments, suggestions } |
| No Authorization header | 401      | Access denied: No token provided         |
| Invalid / expired JWT   | 403      | Access denied: Invalid token             |

*The 401/403 paths are implemented by middleware. A Postman collection or captured run is not committed, so the presentation should execute these three calls live rather than claim pre-recorded proof.*

SECTION 06 · FINAL PROJECT REPORT

Authentication and access control

Google establishes identity a one-day JWT carries the authenticated user context across services.

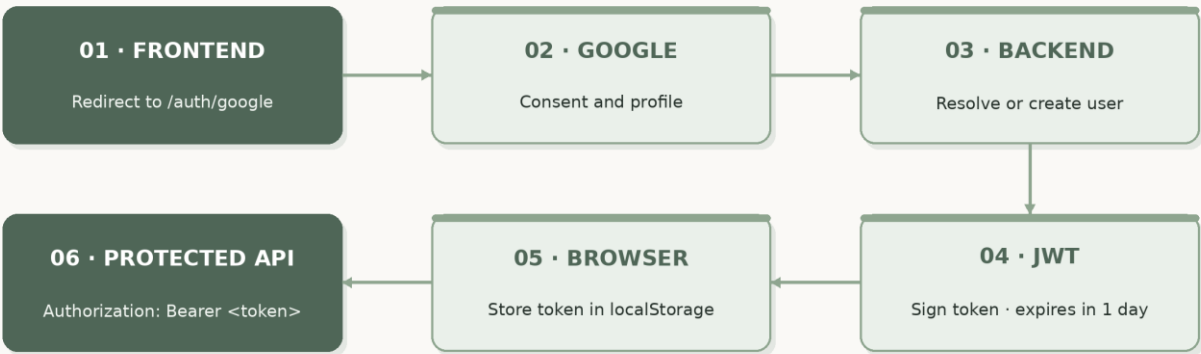


Figure 4. Current OAuth-to-JWT flow and the bearer token used on protected requests.

| Control                | Current behavior                                                           | Status      |
|------------------------|----------------------------------------------------------------------------|-------------|
| Route restriction      | ProtectedRoute redirects a browser without a token to the login screen.    | Implemented |
| Token validation       | Main, document, scoring REST and AI middleware verify JWT signatures.      | Implemented |
| Service context        | Scoring forwards the caller's Authorization header to data owners.         | Implemented |
| Unauthenticated errors | REST middleware distinguishes missing token (401) and invalid token (403). | Implemented |
| GraphQL edge           | Checks token presence but does not verify the signature at entry.          | Limitation  |

**PRODUCTION HARDENING** The OAuth callback currently places the JWT in the redirect query string and Login.jsx stores it in localStorage. A production design should remove fallback secrets, use hardened secure cookies or another reviewed session model, add CSRF defenses, validation, rate limiting and security headers, and enforce consistent authorization at the GraphQL boundary.

SECTION 07 · FINAL PROJECT REPORT

Scoring engine and AI extension

Deterministic decision logic remains authoritative; generative AI improves presentation, not eligibility.



Figure 5. Maximum contribution of each transparent scoring dimension.

| Dimension             | Max | Full-credit condition               |
|-----------------------|-----|-------------------------------------|
| Document completeness | 25  | All five categories supplied        |
| Rent affordability    | 25  | Target rent ≤ 33% of monthly income |
| Guarantor strength    | 25  | Combined guarantor income ≥ 3× rent |
| Profile stability     | 15  | CDI employment status               |
| Profile clarity       | 10  | Core financial fields complete      |
| Total                 | 100 | Capped at 100                       |

The service returns score, missingDocuments and suggestions. Affordability and guarantor rules include partial-credit bands, while the final message classifies the profile as excellent, strong, average or weak. Per-category output and score history are not currently exposed.

Gemini landlord pitch

**Protected request** → forward JWT to retrieve the dossier → insert financial context into a constrained French prompt → call **gemini-2.0-flash** → return a three-sentence pitch. If a configured provider call fails, a profile-aware local template is returned.

**IMPLEMENTATION NOTE** If GEMINI\_API\_KEY is missing or still contains the placeholder, the current route returns HTTP 400. The fallback runs only after a configured Gemini request fails; this is narrower than the README's no-key fallback claim.



## SECTION 08 · FINAL PROJECT REPORT

## Deliverables and rubric compliance

Source code is the authority where older planning documents no longer match the implementation.

| Brief requirement              | Repository evidence                               | Status       |
|--------------------------------|---------------------------------------------------|--------------|
| Interactive JS frontend        | Login, dashboard, dossier and document components | Verified     |
| Main backend + 2 microservices | backend/ + document-service/ + scoring-service/   | Verified     |
| At least 2 databases           | PostgreSQL/Prisma + MongoDB/Mongoose              | Verified     |
| Frontend/API communication     | Fetch calls to ports 5000, 5001 and 5002          | Verified     |
| Authenticated communication    | Bearer JWT + service token forwarding             | Implemented* |
| Google login                   | Passport Google strategy and callback             | Code-ready   |
| Restricted URLs                | Protected dashboard + invalid-token redirect      | Verified     |
| JWT API as a service           | GET /api/v1/dossier-score                         | Verified     |
| JWT / no-JWT demo              | 200 / 401 / 403 paths exist                       | Live demo    |
| Unauthenticated error handling | JWT middleware responses                          | Verified     |
| External API bonus             | Google Gemini pitch route                         | Code-ready   |
| Two API paradigms              | REST + Apollo GraphQL                             | Verified     |
| Team ≤ 3                       | Adam Hamdi, Selim Abouleila, Anaïs Robert         | Verified     |

### Repository deliverables

**Application:** four runnable projects and dependency locks. **Data:** Prisma schema, migration and Docker database stack. **Configuration:** .env.example. **Documentation:** README, six engineering notes and pre-project plan.

**VALIDATION SNAPSHOT** Frontend production build passed; lint passed with five unused-variable warnings; backend and microservice JavaScript syntax checks passed; Prisma schema validation passed. A full Docker-backed integration run was not part of this report audit.

\* GraphQL forwards a token but needs direct edge verification and consistent denial behavior before production.